Version 2.0

January 2026

DATABASE

MODULE 3 - RELATIONAL DATA AND NORMALIZATION

Author:

Dr. Ir. Agus Eko Minarno, M.Kom., IPM.

Naufal Arkaan

Ismail Dwi Muh. Anugerah

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

INTRODUCTION

OBJECTIVES

1. Students understand advanced data relationship concepts in database modeling.
2. Students understand the concept and purpose of data normalization.
3. Students understand the relationship between CDM, table structures, and normalization stages.

MODULE TARGETS

1. Students can apply advanced relationships (supertype–subtype, ARC, recursive, transferable, and non-transferable) in their CDM.
2. Students can perform data normalization from UNF to 3NF using table structures.
3. Students can compare CDM designs before and after normalization and explain the differences.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

   رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Laptop or personal computer.
3. Oracle SQL Developer Data Modeler installed and ready to use.
   LINK-SQL_Developer_Data_Modeler
4. Strong motivation and commitment to learning.

KEYWORDS

Data Relationship, Data Normalization

DATA RELATIONSHIPS

In database design, data relationships describe how entities are connected to each other. Relationships help us understand how data is related and how it should be stored to represent real-world situations correctly.

As a data model becomes more complex, simple relationships such as One-to-One or One-to-Many are sometimes not enough. Some situations require more advanced relationship structures to describe conditions, classifications, ownership, or hierarchy in the data.

In this module, you will learn several types of advanced data relationships, including supertype and subtype, arc relationship, barred relationship, transferable and non-transferable relationship, and recursive relationship. Each of these relationships is used for a specific purpose. The goal is not to memorize symbols, but to understand when and why each relationship should be used.

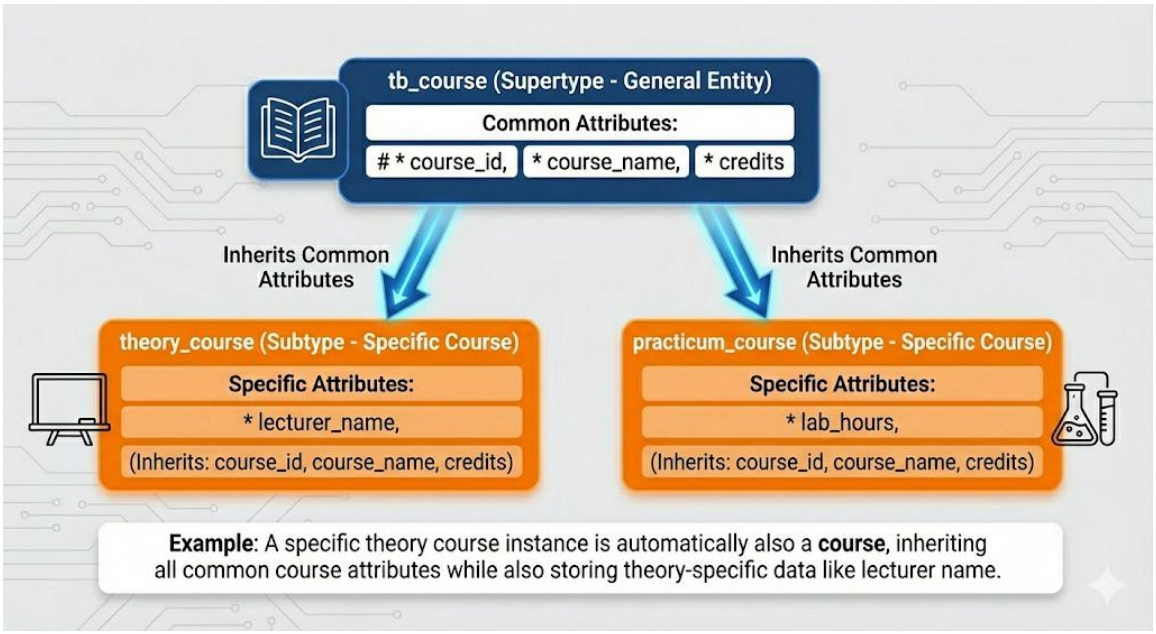
1. SUPERTYPE & SUBTYPE

A supertype and subtype relationship is used when several entities represent variations of the same real-world object. These entities share some common information, but also have specific data that does not apply to all cases. Instead of storing all attributes in one entity or duplicating the same attributes across multiple entities, the data is organized into a general entity and more specific entities.

The **supertype** represents the general entity that stores attributes shared by all data, such as identifiers and basic information. The **subtypes** represent more specific categories and inherit all attributes from the supertype, while also storing attributes that are unique to them. This structure helps reduce duplicated data, avoids many empty (null) values, and makes the data model easier to understand and maintain.

In practice, a subtype cannot exist without its supertype, because every instance of a subtype must also be an instance of the supertype. For this reason, attributes placed in the supertype must apply to all subtypes, while attributes that only apply to certain cases should be placed in the appropriate subtype. Supertype and subtype relationships should only be used when there is a clear conceptual reason, not simply to increase the number of entities in the model.

EXAMPLE:



To illustrate the supertype and subtype concept, consider an academic system that manages course data. In this model, tb_course acts as the supertype, storing attributes that apply to all courses, such as course_id, course_name, and credits.

Two subtypes are derived from this supertype: theory_course and practicum_course. Both subtypes inherit the identifier and common attributes from tb_course, but each subtype also has its own specific attributes. The theory_course subtype includes lecturer_name, while the practicum_course subtype includes lab_hours.

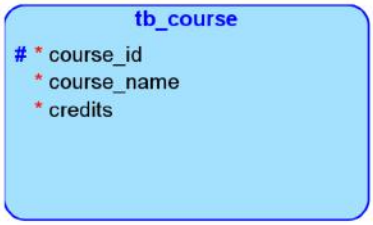
When modeling supertype and subtype in Oracle Data Modeler, students should pay attention to the following important points:

- The identifier of the supertype must be inherited by all subtypes.
- Common attributes should not be duplicated in the subtype entities.
- Subtype entities should only contain attributes that are relevant to that subtype.

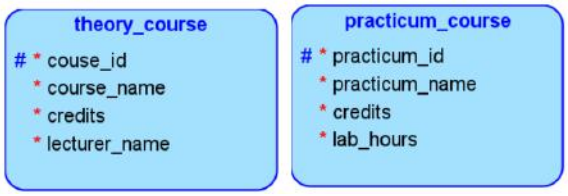
Oracle Data Modeler visually shows how attributes are inherited, making it easier to understand the relationship between the supertype and its subtypes.

HOW TO IMPLEMENT SUPERTYPE AND SUBTYPE IN ORACLE DATA MODELER

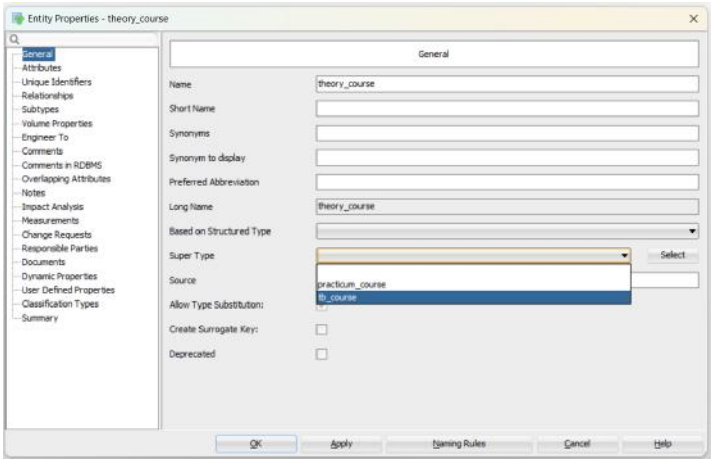
1. The first thing is to determine the supertype and subtype. Example:
   - Supertype: **tb_course**.

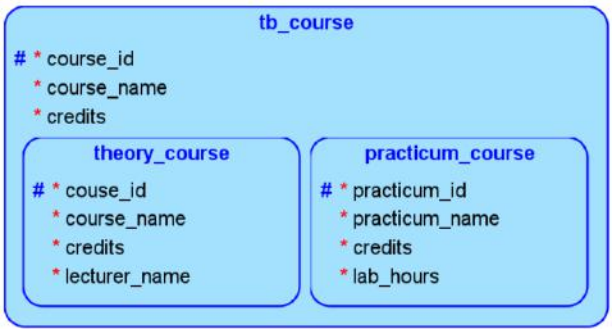


   - Subtype: **theory_course** and **practicum_course**.



2. Create entities along with the attributes of the supertype and subtype that have been determined.
3. Double-click on the subtype entity until the properties window appears.
4. Move to the "General" tab and in the "Supertype" section, select the supertype entity that was determined at the beginning. Then click “Apply” and “OK”.





2. ARC (ARC RELATIONSHIP)

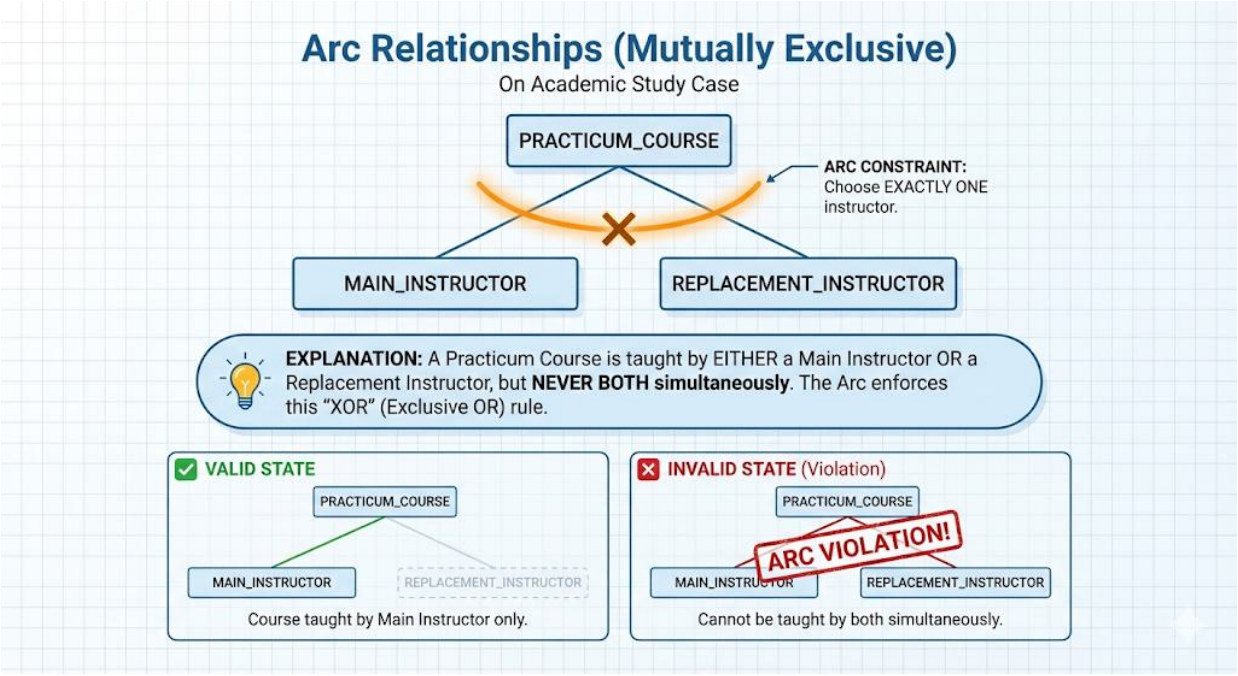
An ARC relationship is used when one entity can be related to several other entities, but only one relationship is allowed to exist at a time. This relationship represents a situation where an entity must choose one option from multiple alternatives, based on a specific rule.

In an academic system, consider a practicum course. A practicum course is explained by one instructor at a time. Two instructors cannot teach the practicum at the same time. If the main instructor cannot attend, another instructor can take over, but only as a replacement, not together with the main instructor.

This situation is best modeled using an ARC relationship because the practicum course can be related to multiple instructors, but only one instructor relationship can be active at a time.

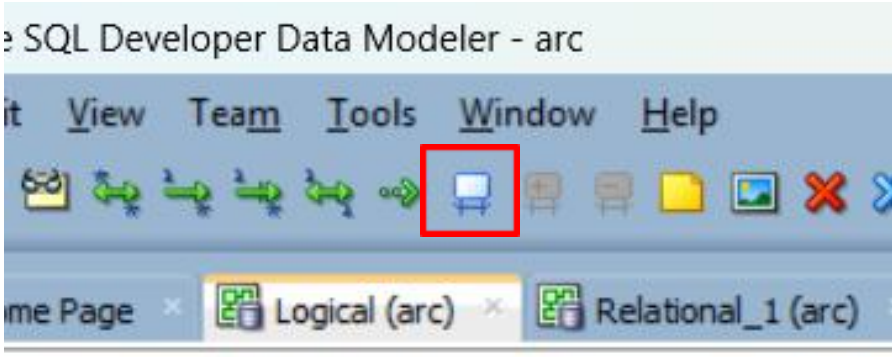
In Oracle Data Modeler, an ARC relationship is represented using a **curved arc symbol** that groups the instructor relationships connected to the practicum course entity. The arc indicates that the relationships are mutually exclusive. When defining an ARC relationship, all relationships inside the arc should be defined consistently and should clearly represent the real-world rule that only one relationship can exist at a time.

ARC relationships should be used only when exclusivity is required. If multiple relationships are allowed to exist at the same time, then an ARC relationship is not appropriate.

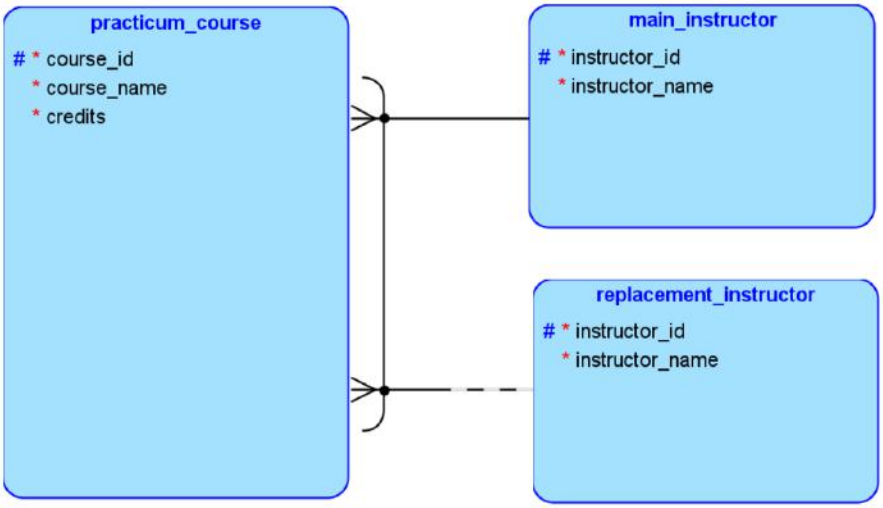


STEPS TO CREATE AN ARC RELATIONSHIP

1. Prepare several relationships, where one relationship is mandatory or both relationships are optional.
2. Press and hold Ctrl, then click the main entity, followed by the first relationship and the second relationship. After that, click the ARC icon.



3. The ARC relationship will appear as shown in the image below.



3. BARRED

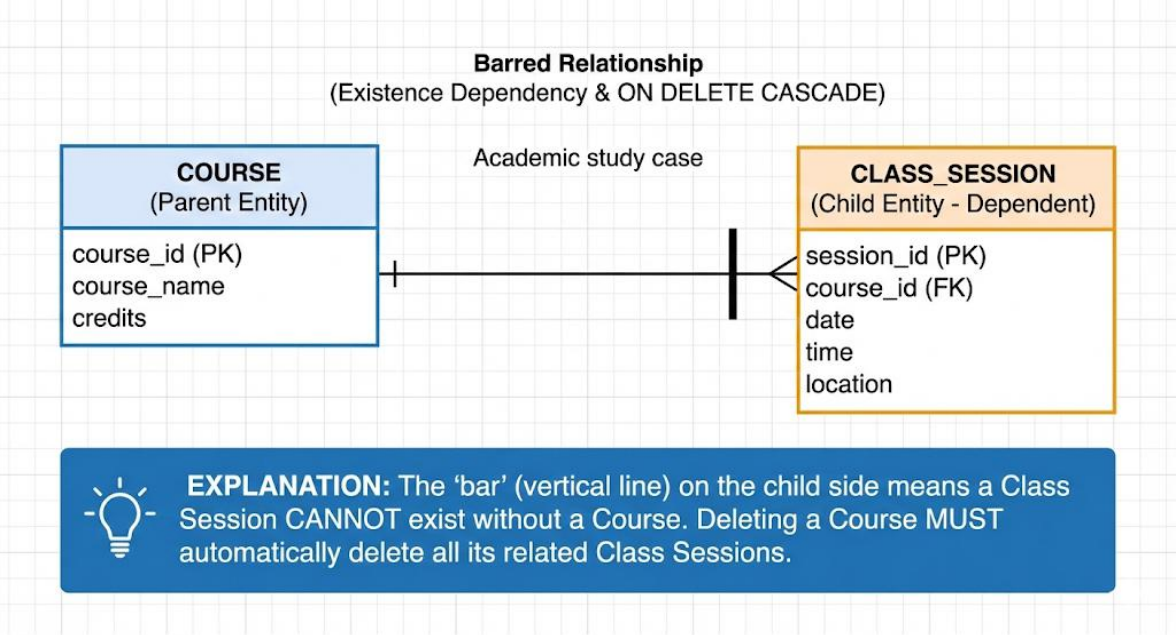
A Barred Relationship is used when one entity cannot exist without another entity. This relationship represents a situation where the child entity is fully dependent on the parent entity. If the parent entity does not exist, the child entity also cannot exist.

In an academic system, consider a course and its class sessions. A class session is created only for a specific course. It is not possible to have a class session without a course. If a course is removed, all related class sessions must also be removed.

This situation is best modeled using a Barred Relationship because the class session depends entirely on the course for its existence. The class session does not have meaning or identity without the course it belongs to.

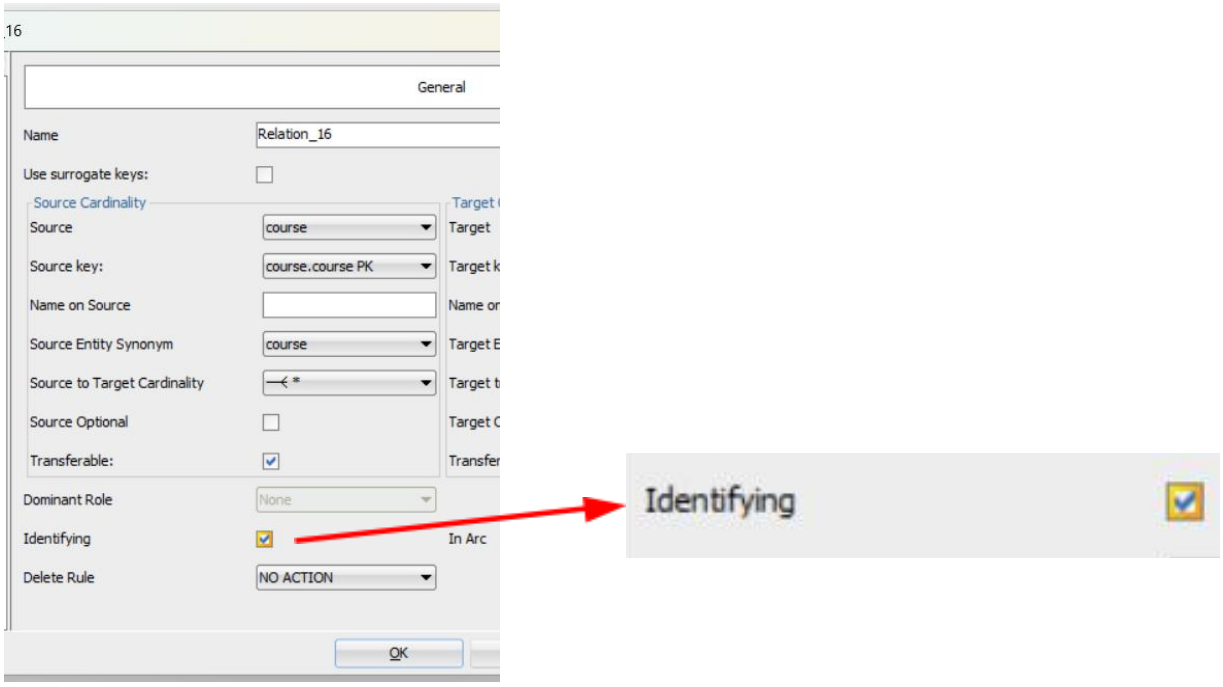
In Oracle Data Modeler, a barred relationship is represented using a **bar symbol** near the child entity side of the relationship line. This symbol shows that the child entity is dependent on the parent entity. In most cases, the identifier of the parent entity becomes part of the identifier of the child entity, showing this dependency clearly.

Barred relationships should be used only when the child entity cannot exist independently. If the child entity can exist without the parent entity, then a barred relationship should not be used.

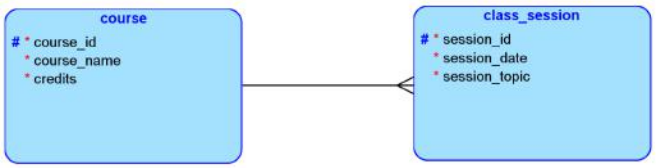
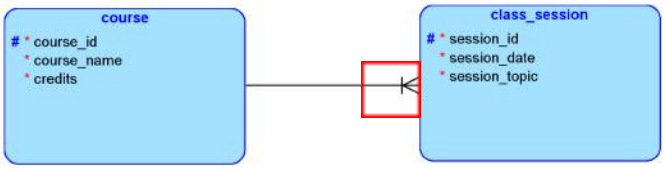


STEPS TO CREATE A BARRED

1. First, prepare the required entities and create a relationship between them.
2. Double-click the relationship that has been created. In the Relationship Properties window, open the “General” tab, then check the “Identifying” option.



3. After that, a bar symbol will appear on the child entity side of the relationship, as shown in the image below.

Without Barred	With Barred
	

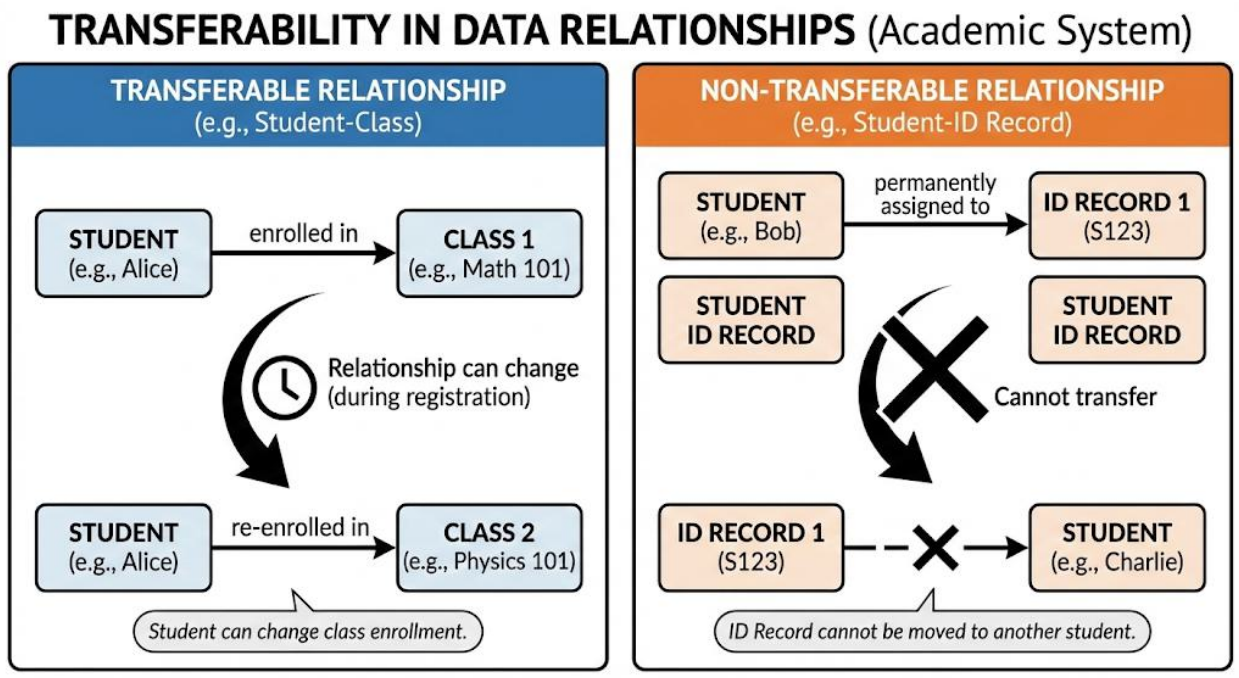
4. TRANSFERABILITY AND NON-TRANSFERABLE RELATIONSHIP

A transferable relationship allows a relationship between two entities to move from one entity to another, while a non-transferable relationship does not allow such movement once the relationship is created. This concept is used to describe whether a relationship is flexible or fixed based on real-world rules.

In an academic system, a student may change their class at any time while the class registration period is still open. This means the relationship between the student and the class can change, making it a transferable relationship. On the other hand, a student ID record is permanently assigned to one student and cannot be transferred to another student. This is an example of a non-transferable relationship.

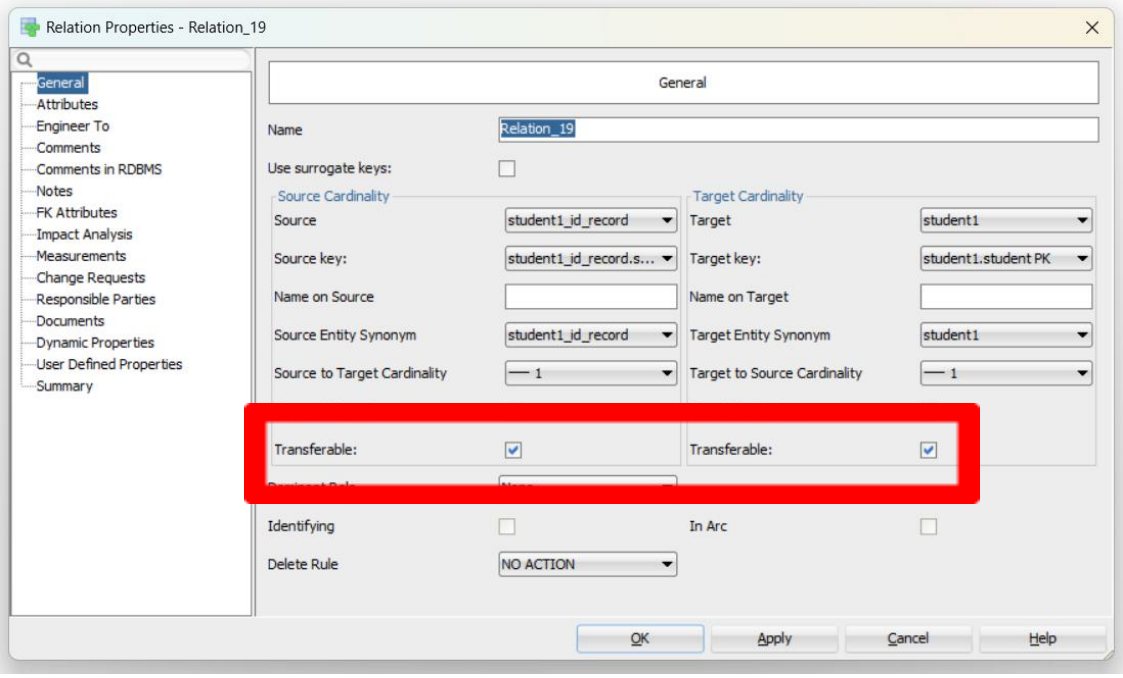
In Oracle Data Modeler, transferability is defined in the relationship properties to reflect whether a relationship is allowed to change. Choosing the correct type is important because it affects how data changes are handled and ensures that the data model follows real-world rules correctly.

Transferable relationships should be used when changes are expected, while non-transferable relationships should be used when the relationship must remain permanent.

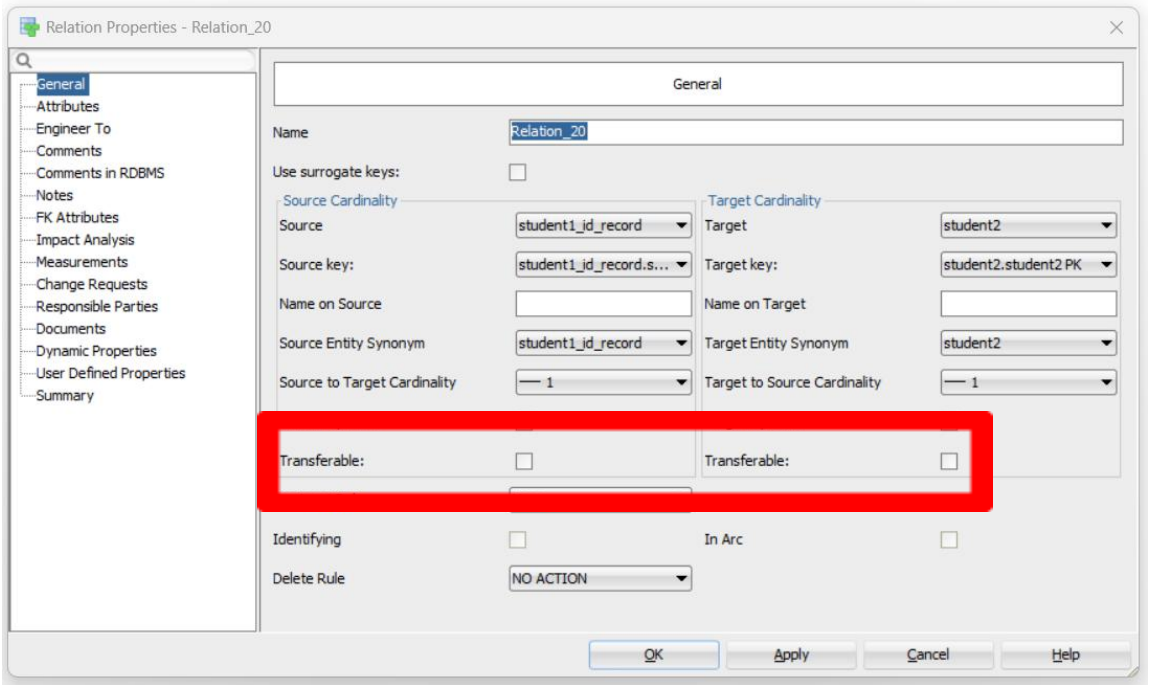


HOW TO IMPLEMENT TRANSFERABILITY IN ORACLE DATA MODELER

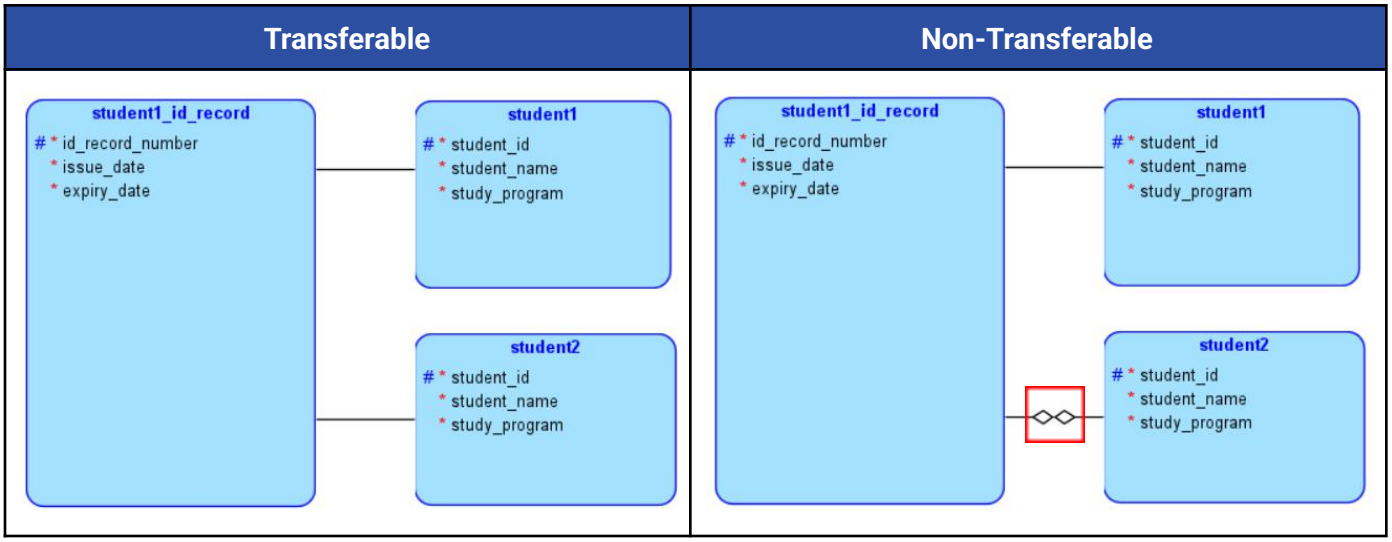
1. First, prepare the required entities and create the relationship between them.
   - Double-click the relationship to open the Relationship Properties window.
   - In the General tab, locate the “Transferable” option.



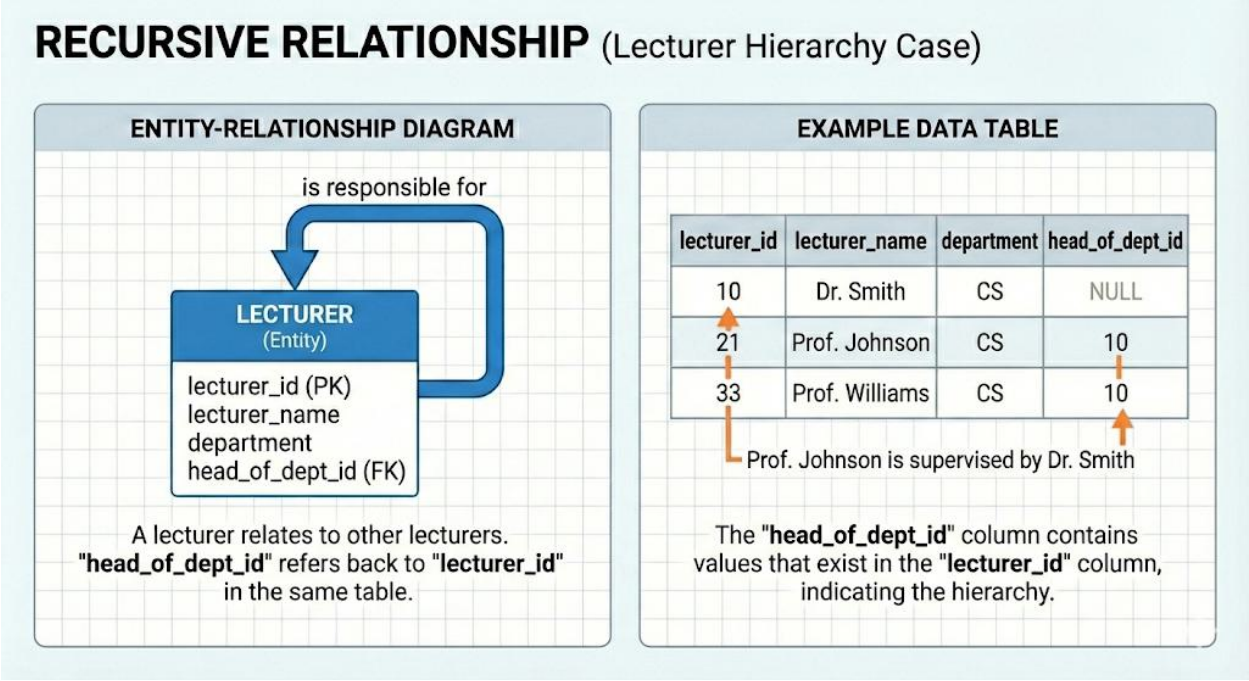
2. If the relationship is non-transferable, leave this option unchecked.



3. After that, the non-transferable symbol will appear on the relationship, as shown in the image below.



5. RECURSIVE RELATIONSHIP



A recursive relationship is used when an entity is related to itself. This means that one record in an entity can be connected to another record in the same entity. This relationship is commonly used to represent data that has a hierarchical structure.

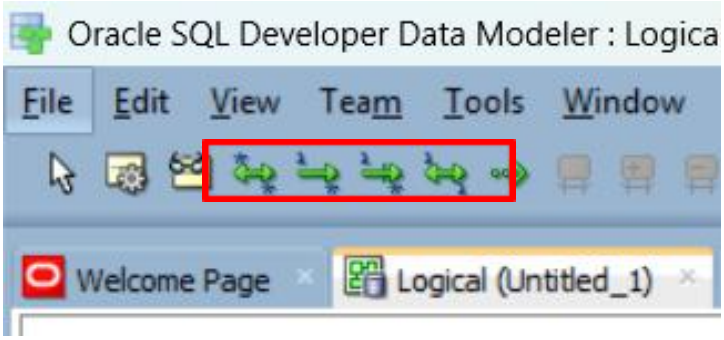
In an academic system, consider a lecturer hierarchy. A lecturer can become the head of a department, and other lecturers can be members of that department. The head of the department is still a lecturer, and the members are also lecturers. They are all stored in the same LECTURER entity. However, one lecturer can be responsible for other lecturers.

In Oracle Data Modeler, a recursive relationship is shown as a relationship line that starts and ends at the same entity. To make the model clear, the roles in the relationship should be named properly, for example lecturer and head of department.

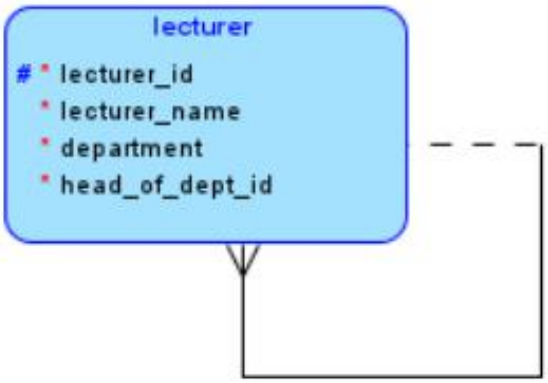
Recursive relationships should be used when data naturally refers to itself, such as prerequisites, organizational structures, or hierarchies. If the relationship involves different entities, then a recursive relationship should not be used.

HOW TO IMPLEMENT RECURSIVE RELATIONSHIP IN ORACLE DATA MODELER

1. First, prepare a single entity.
2. Select the relationship tool that will be used for the recursive relationship.



3. Click the entity where the recursive relationship will be applied, then click the same entity again.
4. A recursive relationship will appear, as shown in the image below.



CDM TO TABLE STRUCTURE

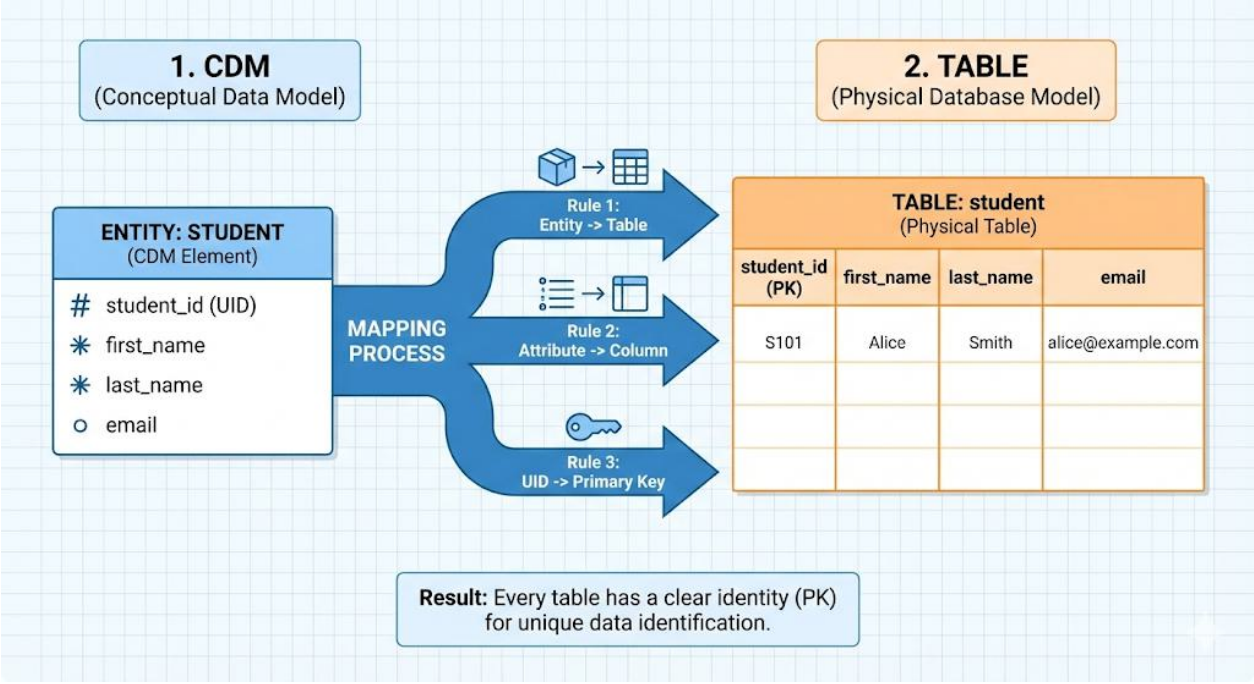
A Conceptual Data Model (CDM) describes data at a high level. It focuses on what data is needed and how data is related, without considering how the data will be physically stored in a database. To implement a database, the CDM must be translated into a table structure, where data is represented in rows and columns.

Understanding how a CDM is converted into tables is very important, especially for the next topic, data normalization. Normalization is easier to understand when data is represented in table form, because duplicated data, repeating values, and data dependencies can be seen more clearly in tables than in a diagram.

1. MAPPING CDM ELEMENTS TO TABLES

Understanding how a CDM is converted into tables is very important because it shows how a conceptual design is turned into a real database structure. This understanding becomes especially useful in the next topic, data normalization, where data problems are easier to identify when information is presented in table form rather than diagrams.

In general, each entity in a CDM is represented as one table, and each attribute becomes a column in that table. The Unique Identifier (UID) of the entity is used as the Primary Key to uniquely identify each record. By using this approach, the table structure stays consistent with the CDM and makes the normalization process easier to understand and apply.

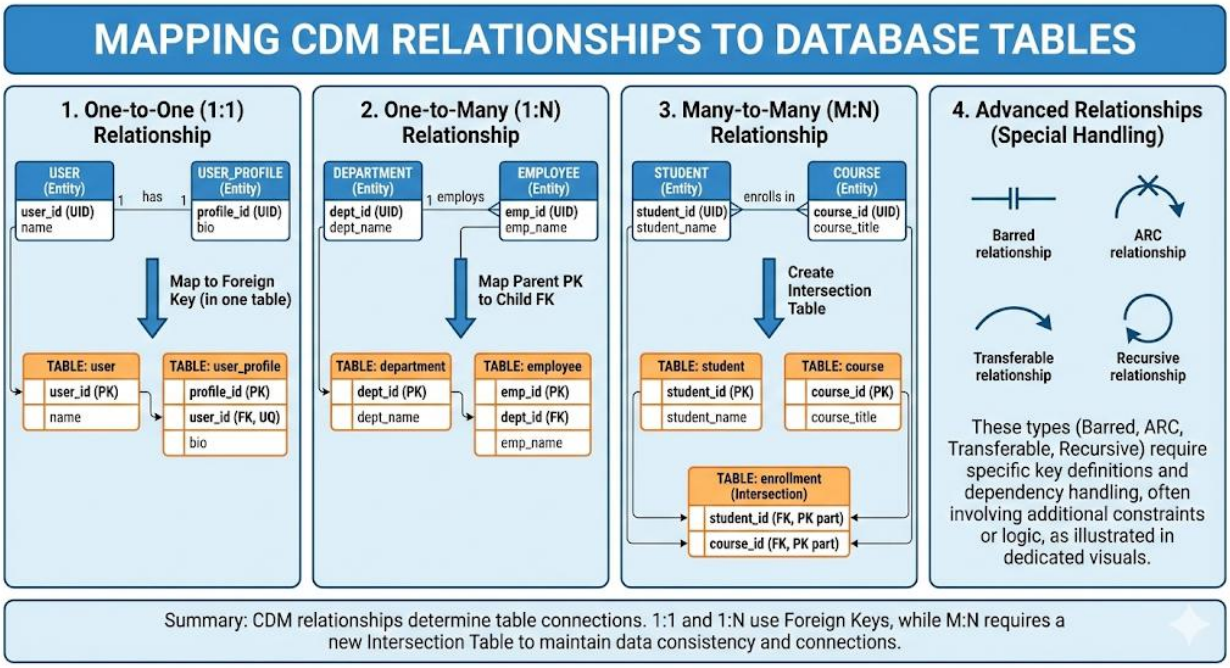


2. MAPPING CDM RELATIONSHIPS TO TABLES

Relationships defined in a CDM determine how tables are connected to each other in the database. When a CDM is converted into tables, relationships are implemented using foreign keys or additional tables, depending on the type of relationship. This step is important because it ensures that data between tables remains connected and consistent.

For a One-to-One (1:1) relationship, a foreign key is placed in one of the tables based on the business rule. In a One-to-Many (1:N) relationship, the primary key of the parent table is added as a foreign key in the child table. This is the most common type of relationship and clearly shows how one record can be linked to multiple records in another table.

For a Many-to-Many (M:N) relationship, a new table called an intersection table is created. This table stores foreign keys from both related tables and represents the relationship between them. Advanced relationships such as barred, ARC, transferable, and recursive relationships also influence table design, especially in how keys are defined and how data dependencies are handled during normalization.



DATA NORMALIZATION

Data normalization is part of database design that helps organize data so it makes sense and is easier to work with. The idea is simple, such as avoiding storing the same information over and over again. When data is arranged properly, the database becomes easier to maintain and less likely to cause problems as it grows.

If normalization is not applied, the same data can appear in many places. This might not seem like a problem at first, but it becomes an issue when changes are needed. Updating one piece of data may require several updates in different records, and missing just one of them can lead to inconsistent data. Normalization helps prevent this by placing each piece of information where it logically belongs.

Normalization also prepares the database for future changes. Data rarely stays the same for a long time, and a good structure makes updates safer and more controlled. With normalization, relationships between data are clearer, maintenance becomes simpler, and the database structure better reflects how data works in real situations.

Normalization is not done all at once. It is applied gradually, where each step fixes a specific problem that was not solved in the previous step. These steps are known as normal forms.

The most commonly used normal forms in database design are:

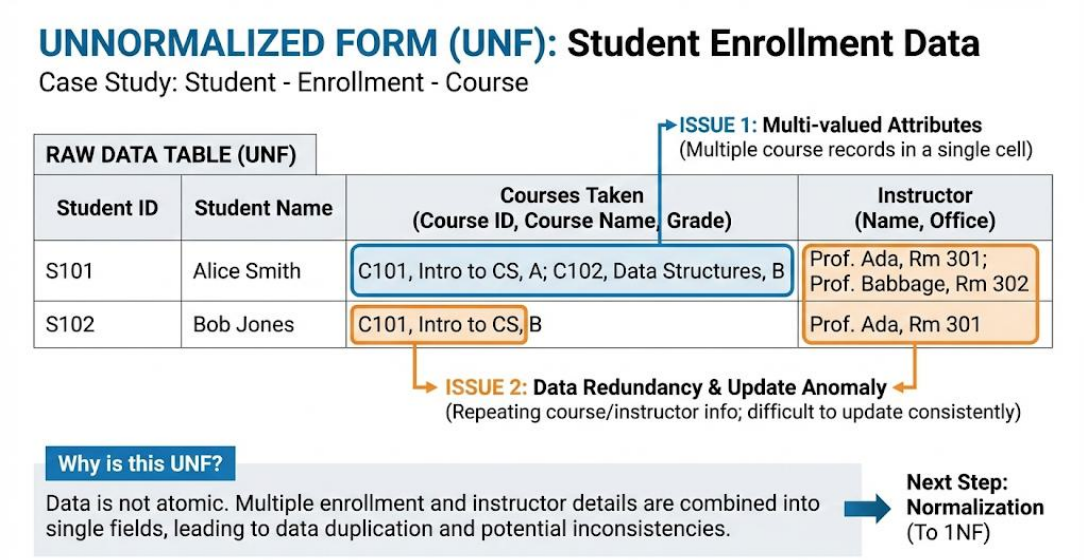
- Unnormalized Form (UNF)
- First Normal Form (1NF)
- Second Normal Form (2NF)
- Third Normal Form (3NF)

1. UNNORMALIZED FORM (UNF)

Unnormalized Form (UNF) is the earliest stage of data organization. At this stage, data is stored in a very basic way, often by combining different types of information into a single table. The main focus is usually just to record data, without considering efficiency or long-term maintenance.

Because of this, the same data may appear many times in different records. This structure can cause problems when data needs to be updated or changed, since one update may require changes in multiple places. For this reason, UNF is only a starting point and not suitable for a proper database design.

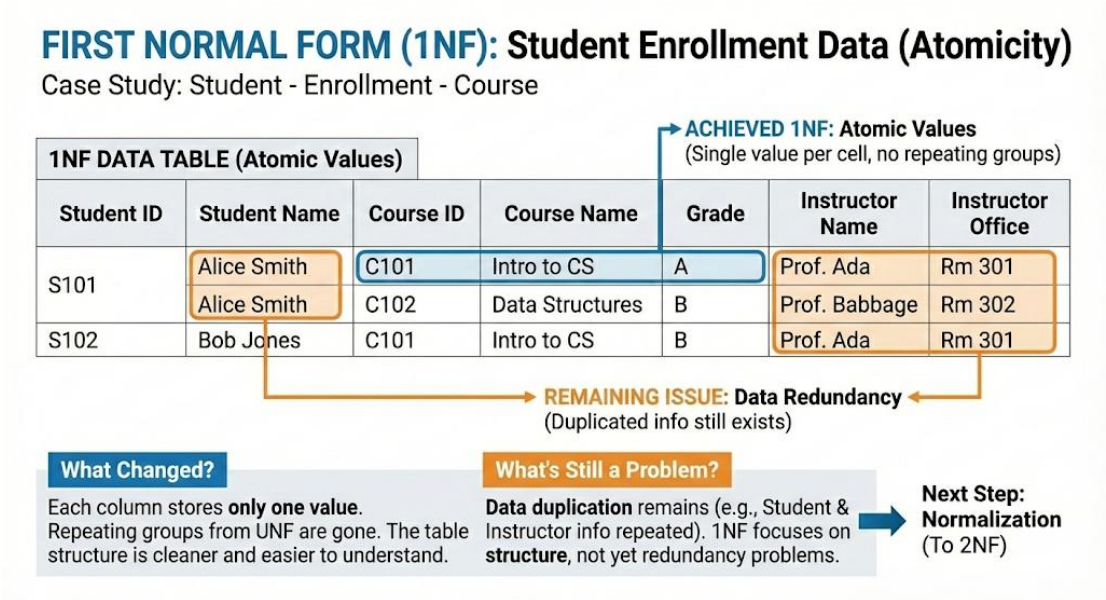
Unnormalized Form (UNF) is not a formal form, but a condition of data before normalization is applied.



2. FIRST NORMAL FORM (1NF)

First Normal Form (1NF) improves the basic structure of the data. In 1NF, each column must store only one value, and repeating groups are removed. This makes the table easier to read and understand.

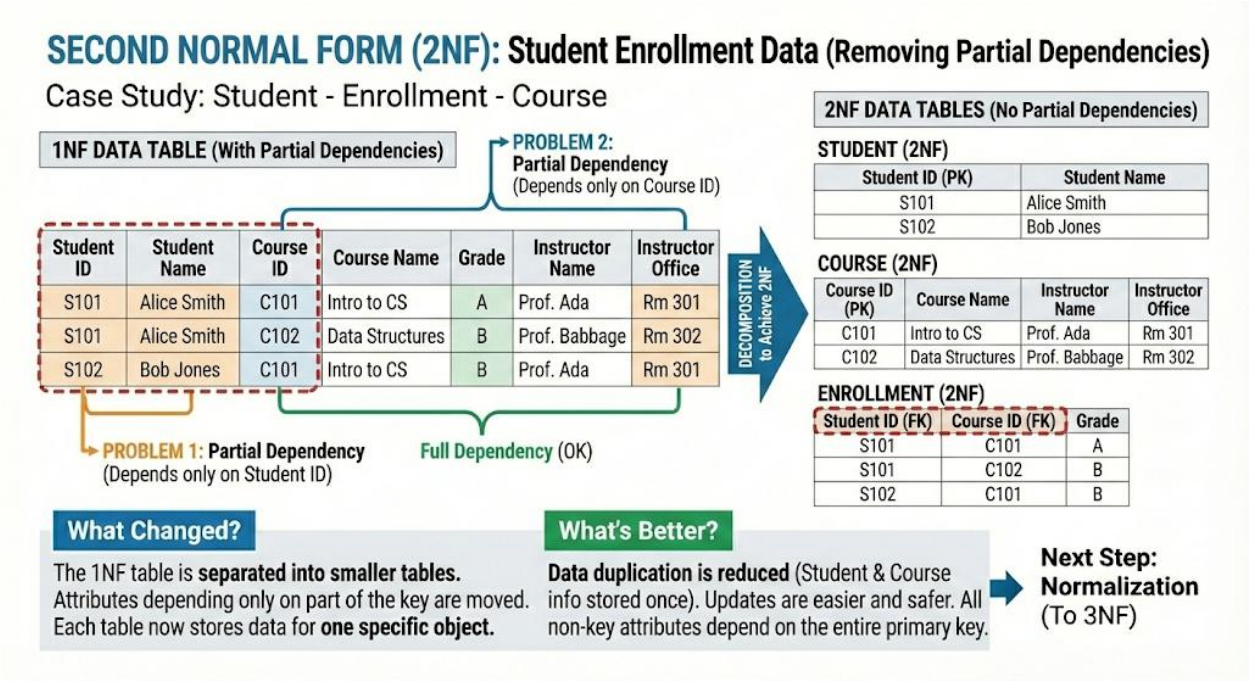
Although the table becomes more organized in 1NF, data duplication can still occur. Information that belongs to different objects may still be stored together. This shows that 1NF focuses on cleaning the table structure, but it does not yet solve redundancy problems.



3. SECOND NORMAL FORM (2NF)

Second Normal Form (2NF) focuses on how data depends on the primary key. A table is in 2NF when all non-key attributes depend on the entire primary key. This is especially important when a table uses a composite key.

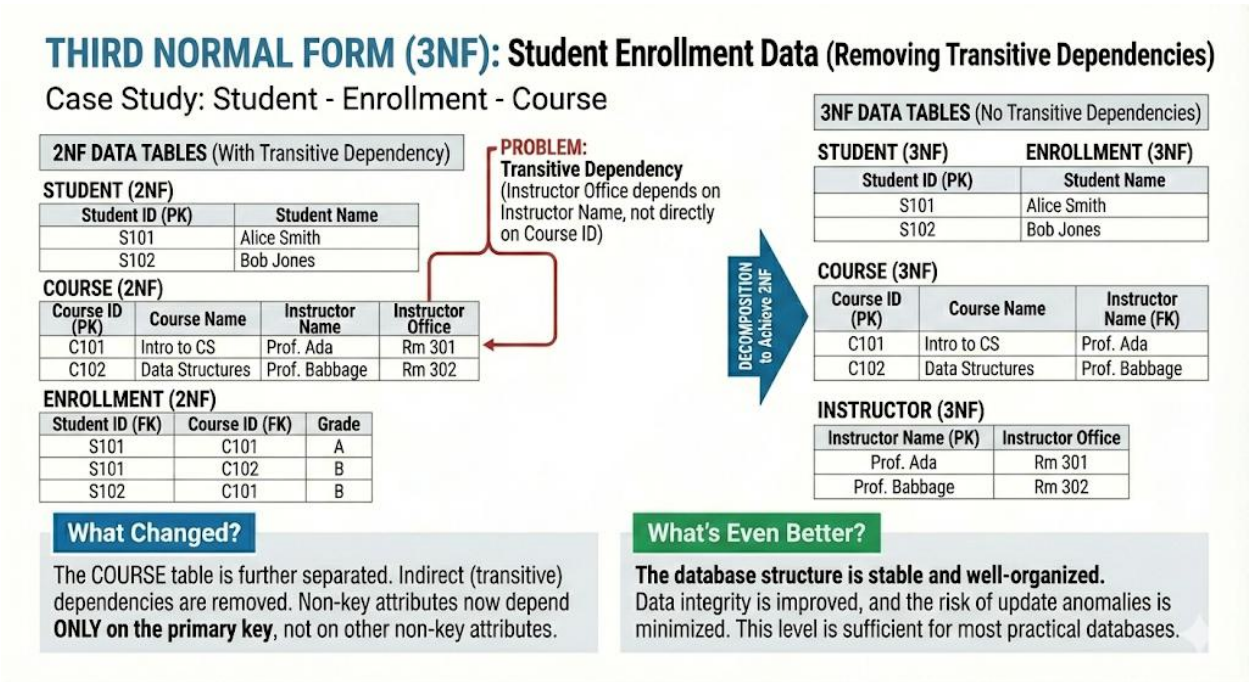
If some attributes depend only on part of the key, the table needs to be separated into smaller tables. By doing this, each table stores data that belongs to one specific object. As a result, data duplication is reduced and updates become easier and safer.



4. THIRD NORMAL FORM (3NF)

Third Normal Form (3NF) further improves the table by removing indirect dependencies between attributes. At this stage, non-key attributes should depend only on the primary key and not on other non-key attributes.

When a table reaches 3NF, the database structure becomes stable and well-organized. Most practical database systems are designed up to this level because it provides a good balance between simplicity and data integrity.



5. CONCLUSION

STEPS	DESCRIPTION
UNF	Data is stored in a raw and unstructured form. Multiple pieces of information may be combined in one table, including repeating groups or attributes with multiple values in a single column. This causes data duplication and makes updates difficult. Because of these issues, the table must be restructured into 1NF to ensure each attribute holds only a single value.
1NF	The table structure is improved so that each column contains only one value and repeating groups are removed. The data becomes easier to read and process. However, redundant data still exists because attributes related to different entities may still be stored together. This leads to partial dependency problems, which is why further normalization to 2NF is required.
2NF	Data is separated so that all non-key attributes depend on the entire primary key. This removes partial dependencies and reduces duplication. Even so, some attributes may still depend indirectly on other non-key attributes. To eliminate these transitive dependencies, the table must be normalized further into 3NF.
3NF	All transitive dependencies are removed. Each non-key attribute depends only on the primary key and nothing else. The table structure becomes stable, consistent, and easier to maintain. At this stage, the data design is considered suitable for implementation and for rebuilding a cleaner CDM.

PRACTICE ACTIVITY

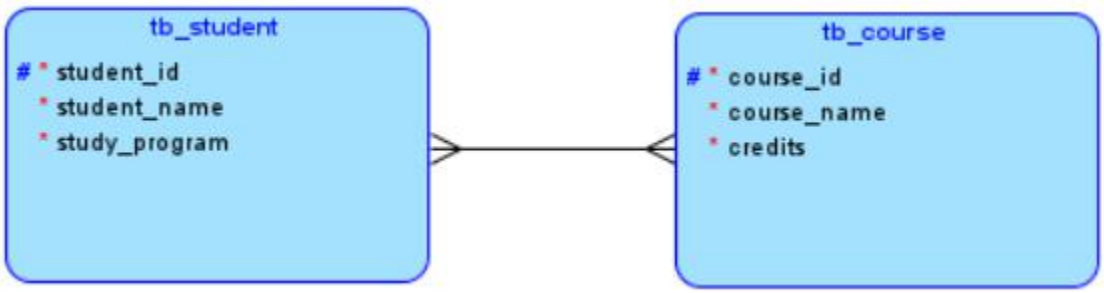
EXAMPLE OF NORMALIZING DATA FROM A CDM

This case study shows how a Conceptual Data Model (CDM) is translated into tables and then improved step by step using data normalization. The data shown in the tables is **dummy data**, created only to help explain the concepts.

Converting a CDM into Tables

Before normalization can be applied, the CDM is first converted into a table structure. In practice, data normalization is usually performed using tables, because data duplication and dependency are easier to see in rows and columns than in diagrams. At this stage, the table does not need to be perfect. Its purpose is simply to show how data is stored so that normalization can be applied in the next steps.

FROM THIS:



TO THIS:

student_id	student_name	study_program	courses
S001	Rei	Informatics	C101-Database(3), C102-Data Structure(3)
S002	Shinji	Informatics	C101-Database(3)

At first, this table looks practical because all information is stored in one place. However, what if a student changes their name or study program?

The same data appears in several rows, so multiple updates are required. If one row is missed, the data becomes inconsistent. The same issue occurs with course information. This is why the table is still considered unnormalized because data is mixed together, repeated frequently, and difficult to maintain.

1NF – First Normal Form

The first step of normalization is First Normal Form (1NF). A table is in 1NF when each column contains a single value, and there are no repeating groups. In this case study, each column already stores only one value. Because of that, the table technically meets the requirement of 1NF.

student_id	student_name	study_program	course_id	course_name	credits
S001	Rei	Informatics	C101	Database	3
S001	Rei	Informatics	C102	Data Structure	3
S002	Shinji	Informatics	C101	Database	3

Structurally, the data is now clearly arranged into rows and columns. However, the main problem remains: student and course data is still duplicated. This shows that 1NF improves the format of the data, but it does not solve redundancy problems.

2NF – Second Normal Form

Second Normal Form focuses on removing partial dependency. In this case, the primary key of the table is a combination of student ID and course ID. However, some attributes depend only on one part of that key. Student-related data depends only on the student, and course-related data depends only on the course.

To solve this problem, the table is split into separate tables for students, courses, and enrollments. After this change, each table stores data that fully depends on its own primary key. This greatly reduces duplicated data and makes updates safer and more consistent.

Table Student

student_id	student_name	study_program
S001	Rei	Informatics
S001	Rei	Informatics
S002	Shinji	Informatics

Table Course

course_id	course_name	credits
C101	Database	3
C102	Data Structure	3
C101	Database	3

Table Enrollment

student_id	course_id
S001	C101
S001	C102
S002	C101

3NF – Third Normal Form

Third Normal Form aims to remove transitive dependency, where one non-key attribute depends on another non-key attribute. In the example, faculty information depends on the study program, not directly on the student. Keeping this data in the student table would cause unnecessary duplication.

By separating study program information into its own table, the structure becomes cleaner and more logical. After reaching 3NF, each table contains data that belongs only to that entity, and the overall design becomes stable, easy to maintain, and suitable for real database implementation.

Table Student

student_id	student_name	study_program
S001	Rei	Informatics
S001	Rei	Informatics
S002	Shinji	Informatics

Table Study Program

study_program	faculty_name
Informatics	Engineering
Informatics	Engineering
Informatics	Engineering

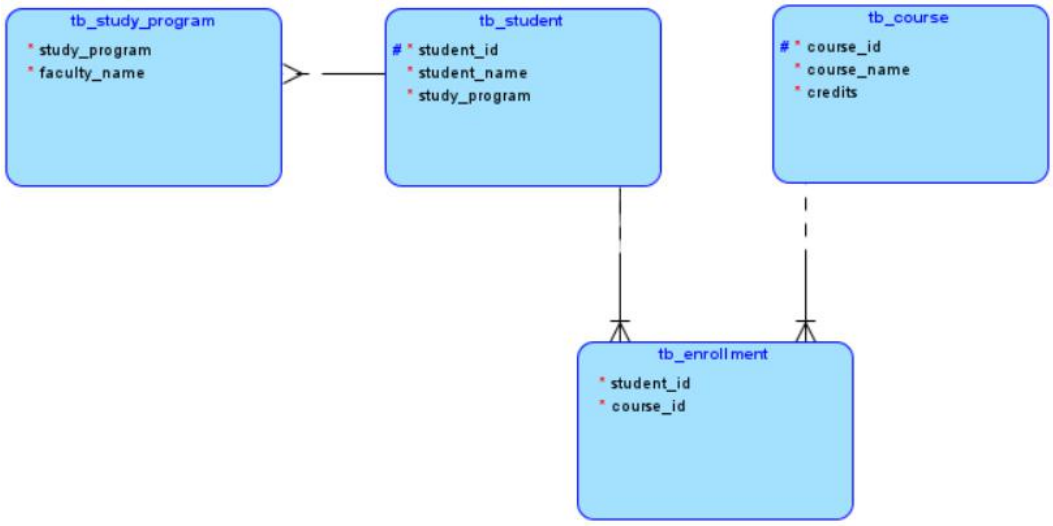
Table Course

course_id	course_name	credits
C101	Database	3
C102	Data Structure	3
C101	Database	3

Table Enrollment

student_id	course_id
S001	C101
S001	C102
S002	C101

Converting Tables into CDM



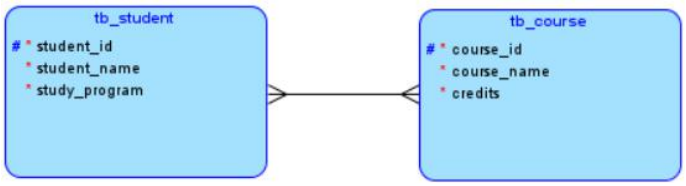
In the final CDM, the relationship between student and enrollment shows that a student may take several courses, while each enrollment record must always be linked to one student. A student can exist in the system even if they have not enrolled in any course yet, but an enrollment record cannot exist without a student. This dependency makes the relationship identifying, because the enrollment entity relies on the student for its identity.

A similar rule applies to the relationship between course and enrollment. One course can be taken by many students, but each enrollment record must refer to exactly one course. A course may exist without any enrollment, for example when it has just been created, but an enrollment record cannot exist without a course. This also creates a strong dependency between the enrollment entity and the course entity.

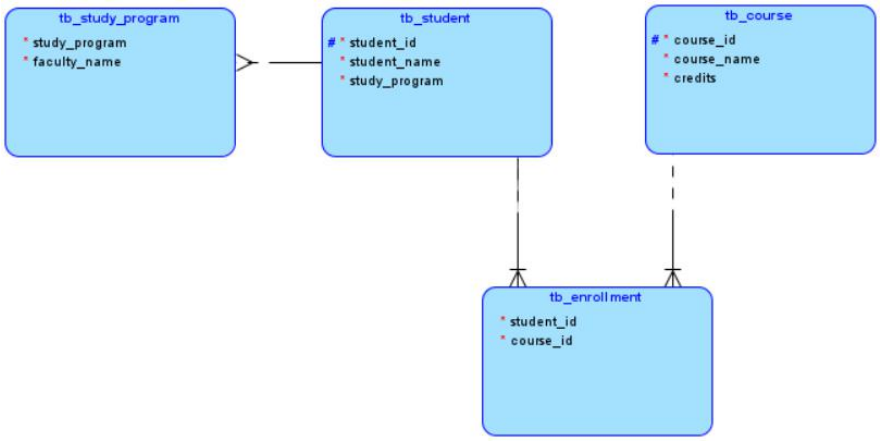
The relationship between study_program and student is different. One study program can have many students, and every student must belong to one study program. However, a student entity can exist independently and does not use the study program identifier as part of its identity. Because of this, the relationship is non-identifying and represents a logical grouping rather than a dependency.

The Comparison

FROM THIS:



TO THIS (AFTER NORMALIZATION):



TRY OUT

In this try out, you are required to continue and extend the CDM that you have developed in the previous module. The CDM must still follow the **Online Shop** theme used earlier.

TRY OUT INSTRUCTIONS

1. Use the CDM from the previous module as the base of your design.
   Your model must still represent an **Online Shop** system.
2. Extend your CDM by implementing advanced data relationships, based on real-world rules in an Online Shop context. These relationships may include:
   - supertype and subtype,
   - ARC relationship,
   - barred (identifying) relationship,
   - transferable or non-transferable relationship,
   - recursive relationship.
3. Apply the selected relationships to existing entities from your previous CDM whenever possible. If required, you may add new entities to support the relationship design.
4. For each relationship you create, clearly define:
   - the relationship type (1:1, 1:M, or M:N),
   - optionality (mandatory or optional),
   - and whether the relationship is identifying or non-identifying, if applicable.
5. Make sure all relationships reflect logical real-world rules in an Online Shop system and are consistent with the data design.

OUTPUT REQUIREMENTS

One CDM diagram that shows:

- the original entities from the previous module,
- the applied advanced relationships,
- any additional entities that support the design.

ASSIGNMENT

In this assignment, you are required to continue using **your own schema** that has been developed since the previous modules. Do not create a new schema. All work in this module must be based on the same CDM and theme you have used before.

In this module, you will:

- extend your existing CDM by applying advanced relationships, and
- perform data normalization based on your own schema and CDM.

The normalization process must follow the same steps demonstrated in the **Practice Activity** section.

ASSIGNMENT INSTRUCTIONS

1. Use the CDM from the previous modules as the base of your assignment.
2. Extend your CDM by applying advanced data relationships, such as:
   - supertype and subtype,
   - ARC relationship,
   - barred (identifying) relationship,
   - transferable or non-transferable relationship,
   - recursive relationship.
3. Convert your updated CDM into a table structure. The tables may be created using Microsoft Excel, spreadsheet tools, Microsoft Word tables, or Google Docs. The table will be used as the starting point for normalization.
4. Perform data normalization step by step, starting from:
   - Unnormalized Form (UNF),
   - First Normal Form (1NF),
   - Second Normal Form (2NF),
   - Third Normal Form (3NF).
5. The data used in your tables may be dummy data, created freely according to your schema and attributes. Make sure the data is consistent with the columns and relationships you designed.
6. After reaching 3NF, convert the normalized tables back into a new CDM.
7. Create a comparison between:
   - the CDM before normalization, and
   - the CDM after normalization.

   Clearly show the differences in entities, attributes, and relationships.

NB:

1. **You must use your own schema consistently from previous modules.**
2. **Dummy data is allowed and encouraged to help explain the normalization process.**
3. **In some cases, based on your own schema and CDM, data normalization beyond a certain stage may not be required. If this happens, you are required to explain clearly to the assistant why your schema does not need further normalization, based on the normalization rules discussed in this module.**

ASSESSMENT

1. Choose one advanced relationship that you applied in your CDM (supertype–subtype, ARC, recursive, transferable, or non-transferable). Explain the real-world rule behind this relationship and why it is appropriate for your schema.
2. If you implemented a supertype–subtype structure, explain:
   - why the supertype was needed, and
   - what makes the subtype entities different from each other.
   - If you did not use supertype–subtype, explain why it was not suitable for your design.
3. If you implemented an ARC relationship, explain:
   - which relationships are included in the ARC, and
   - why only one of them is allowed to exist at a time.
   - If you did not use ARC relationship, explain why it was not suitable for your design.
4. Explain how you performed data normalization from UNF to 3NF. Briefly describe what problem was solved at each stage (UNF, 1NF, 2NF, and 3NF).
5. Compare the CDM before normalization and the CDM after normalization. Explain what changes occurred and why the normalized CDM is more suitable for database implementation.

PRACTICUM GRADING RUBRIC

Assessment Aspects	Points
Try Out	**Total 5%**
• CDM applies at least four (4) different advanced relationship types from this module • Cardinality is defined correctly on both sides of each relationship. • Optionality (mandatory / optional) is defined correctly on both sides. • Supertype and subtype structure (if used) is clear, with attributes placed appropriately. • ARC relationship (if used) correctly represents mutually exclusive relationships. • CDM is clear and consistent.	100
• CDM applies three (3) different advanced relationship types. • Cardinality is defined in most relationships, with minor issues. • Optionality is defined but contains small inconsistencies. • Supertype/subtype or ARC structure exists but lacks clarity in some parts.	80

• CDM applies two (2) advanced relationship types. • Cardinality or optionality is applied inconsistently. • Advanced relationship concepts are present but not clearly represented.	70
• CDM applies only one (1) advanced relationship type, or • Multiple advanced relationships are attempted but mostly incorrect. • Cardinality and optionality are largely missing or incorrect.	60
• CDM shows an attempt to apply advanced relationships, but relationships are unclear, and advanced relationship rules are not properly applied.	40
• No advanced relationships are applied, or • Try Out is not submitted.	0
Assignment	**Total 30%**
• Assignment continues consistently from the schema used in previous modules. • CDM includes advanced relationships applied correctly. • CDM is converted into tables clearly and correctly. • The normalization process is shown step by step from UNF to 3NF. • Each normalization stage (UNF, 1NF, 2NF, 3NF) is clearly differentiated. • Tables at each stage reflect correct normalization rules. • Normalized tables are converted back into a final CDM. • Clear comparison is provided between CDM before normalization, and CDM after normalization. • **If the student's CDM and table structure do not require further normalization. The student must be able to clearly explain why normalization beyond a certain stage is not necessary, based on correct normalization rules and supported by their schema and table structure.**	100
• Assignment continues from previous schema with minor inconsistencies. • Advanced relationships are applied but not fully optimal. • Normalization reaches 3NF, but explanations or table separation lack detail. • Comparison between CDMs exists but is not fully explained.	80
• Assignment follows the required process but shows limited understanding. • Normalization stops at 2NF or 3NF is attempted but incorrect. • Differences between normalization stages are unclear. • CDM comparison is incomplete or weak.	70

• Assignment is submitted but normalization steps are unclear or incorrect, or CDM and table structures are poorly connected. • Advanced relationships are present but incorrectly applied.	60
• Assignment shows an attempt to perform normalization, but stages are not clearly shown, and CDM before and after normalization is missing or invalid. • Understanding of normalization is minimal.	40
• Assignment is not submitted, or • Work does not follow the required process at all.	0
Assessment	**Total 65%**
• Students clearly explain advanced relationships used in their CDM and the real-world rules behind them. • Students correctly explain supertype–subtype, ARC, recursive, and other applied relationships based on their own schema. • The student explains the normalization process from UNF to 3NF clearly and in order. • Students can explain what problem is solved at each normalization stage. • The student clearly compares CDM before normalization and CDM after normalization, and explains why the final CDM is better. • Answers are clear, consistent, and fully based on the student's own work.	100
• Students explain most advanced relationships correctly, but with minor gaps. • Normalization process is explained up to 3NF, but some stages lack detail. • CDM comparison is present but explanation is not fully clear. • Overall understanding is good, with small inaccuracies.	80
• Students understand the general idea of advanced relationships and normalization. • Can mention normalization stages but struggles to explain differences between UNF, 1NF, 2NF, and 3NF. • Comparison between CDMs is weak or incomplete. • Explanations rely more on description than reasoning.	70
• Students answer assessment questions but show limited understanding. • Normalization stages are mentioned but mostly incorrect or unclear. • Advanced relationships are mentioned without clear justification. • CDM comparison is minimal or inaccurate.	60

• Students attend demo week and attempt to answer questions. • Answers are mostly incorrect or very incomplete. • Shows minimal understanding of advanced relationships and normalization concepts.	40
• Student does not attend demo week, or • Students cannot answer assessment questions at all.	0

NB:

1. **If a student attends the material week and participates in the demo, but can only answer assessment questions minimally, the minimum assessment score is 40.**
2. **If a student does not attend, does not submit the try out and assignment, and cannot answer assessment questions, the score is 0.**
3. **In some cases, based on the student's schema and CDM, data normalization beyond a certain stage may not be necessary. If this happens, the student must explain clearly to the assistant why their schema does not require further normalization, based on normalization rules (UNF, 1NF, 2NF, or 3NF). The explanation will be assessed as part of the assignment evaluation.**